



CSC4202

DESIGN AND ANALYSIS OF ALGORITHMS

Group 2

**FLOOD EMERGENCY DELIVERY ROUTE OPTIMIZATION
USING DIJKSTRA'S ALGORITHM**

LECTURER: DR. NUR ARZILAWATI BINTI MD YUNUS

NAME	MATRIC NUMBER
NG YU HUI	223821
CHAI SHUN YING	224414
CHOO YU XUAN	SA00012
SIO ZIXIN	SA00287

1.0 Introduction

1.1 Scenario Background

During severe Northeast Monsoon seasons, flooding along the Langat River can drastically disrupt transportation networks across lower Selangor. Roads that normally support daily transportation may become submerged, while small elevated plantation roads often become the only accessible routes for emergency vehicles.

In this scenario, a major flood event has isolated seven residential areas within the Kuala Langat district: Kampung Bukit Changgang, Kampung Labohan Dagang, Kampung Olak Lempit, Kampung Sri Cheeding, Kampung Sungai Kelambu, Kampung Jenjarom, and Kampung Rancangan Tanah Belia. Emergency relief operations are coordinated from the State Disaster Relief Hub located in Shah Alam, which serves as the main distribution center for critical supplies.

Table 1.1.1: Node Representation

Node ID	Location	Type
0	State Disaster Relief Hub located in Shah Alam	Warehouse
1	Kampung Bukit Changgang	Village A
2	Kampung Labohan Dagang	Village B
3	Kampung Olak Lempit	Village C
4	Kampung Sri Cheeding	Village D
5	Kampung Sungai Kelambu	Village E
6	Kampung Jenjarom	Village F
7	Kampung Rancangan Tanah Belia	Village G

Although the villages are not completely cut off, the road network connecting them has been severely affected. Flood conditions significantly reduce travel efficiency in several ways. First, waterlogged roads greatly reduce vehicle speeds. For example, a 12 km section of the B18 road that normally allows travel speeds of 80 km/h may only support speeds of approximately 15 km/h when covered by 0.4 meters of floodwater. Second, damaged infrastructure can force vehicles onto much longer alternative routes. A collapsed culvert on the direct road to Kampung Labohan Dagang requires relief vehicles to travel through a 24 km plantation detour instead. Third, certain routes contain severe congestion points. Access to Kampung Sungai Kelambu

requires vehicles to pass through Kampung Olak Lempit, where evacuation activities and livestock movement create significant traffic delays.

At the Shah Alam warehouse, essential emergency supplies such as insulin, antibiotics, snake antivenom, and water purification kits are waiting to be delivered. With limited daylight and multiple isolated communities requiring assistance, efficient route planning becomes critical to ensuring that aid reaches affected residents as quickly as possible.

1.2 Problem Statement

Under normal circumstances, logistics planning often assumes that the shortest distance between two locations is also the fastest route. However, this assumption becomes unreliable during flood disasters.

A flooded 8 km paved road may require significantly more travel time than a dry 19 km gravel route located on higher ground. As flood conditions change road accessibility and vehicle speeds, distance alone can no longer be used as an accurate measure of travel efficiency. Instead, factors such as water depth, road damage, congestion, and detours must also be considered.

Human dispatchers working under stressful emergency conditions often rely on their experience and familiarity with local roads when making routing decisions. While this approach may be effective for a single destination, it becomes increasingly difficult when multiple villages require simultaneous assistance.

Table 1.2.1: Traditional and Disaster Response Dispatch Logic

Traditional Dispatch Logic	Distance (km) → Estimated Travel Time
Disaster Response Dispatch Logic	Distance + Road Conditions + Congestion → Actual Travel Time

This situation creates two major challenges for the Selangor State Disaster Management Unit.

The first challenge is the failure of conventional navigation systems. Applications such as Google Maps and Waze rely heavily on traffic patterns and user activity. During floods, roads with no active traffic may incorrectly appear accessible, potentially directing emergency vehicles onto dangerous or impassable routes. This will further delay the delivery of essential emergency supplies to the villages.

The second challenge involves multi destination route planning. Determining the safest route to one village may be manageable, but identifying the most efficient delivery routes to seven

separate villages connected by dozens of partially affected roads quickly becomes too complex for manual decision making. The number of possible route combinations increases dramatically where this has made it difficult for human operators to consistently identify the best solution.

Without a structured mathematical approach, emergency dispatch operations risk becoming inefficient, resulting in delays, wasted resources and reduced effectiveness of relief efforts.

1.3 Need for Algorithmic Optimization

To overcome these challenges, this study proposes the use of Dijkstra's Shortest Path Algorithm as a systematic method for optimizing emergency supply routes. The need for algorithmic routing is driven by three important operational realities.

1.3.1 Time Critical Emergency Response

In disaster situations, every hour can significantly affect health outcomes. For example, a flood victim who receives medical treatment within the first few hours of injury is far more likely to recover successfully than someone whose treatment is delayed for several additional hours. Therefore, route optimization is not simply about reducing travel time; it directly contributes to saving lives and reducing medical complications.

1.3.2. Limited Emergency Transport Resources

Disaster response agencies operate with a limited number of specialized vehicles such as four-wheel drives, amphibious vehicles, and emergency transport trucks. When a vehicle is assigned to an inefficient route, it remains unavailable for other missions for an extended period. By identifying the fastest available paths, route optimization increases vehicle utilization and effectively expands operational capacity without requiring additional assets.

For example, reducing a delivery mission from four hours to less than two hours allows the same vehicle to complete multiple trips within a single day, significantly improving the overall efficiency of relief operations.

1.3.3. Prevent Costly Routing Mistakes

Flooded roads often create situations where drivers are forced to turn back after discovering that a route is impassable. Such incidents result in lost time, wasted fuel and delayed deliveries.

Dijkstra's algorithm minimizes this risk by evaluating all possible routes mathematically before a vehicle is dispatched. Instead of relying on trial and error, relief vehicles are guided along the lowest-cost route based on actual travel conditions, reducing the likelihood of unnecessary detours and failed journeys.

1.4 Objective

The main objective of this study is to transform flood relief logistics in Selangor from a largely experience-based operation into a structured, data-driven decision-making framework using graph theory and route optimization techniques. Specifically, this study aims to:

1. Develop a mathematical model of the flood-affected transportation network by converting factors such as water depth, road conditions, and traffic congestion into a unified travel time cost metric within an undirected graph consisting of eight nodes.
2. Implement Dijkstra's Shortest Path Algorithm to determine and visualize the minimum time delivery routes from the warehouse to all 7 affected villages.
3. Evaluate the effectiveness of algorithmic route planning by comparing Dijkstra generated routes with conventional shortest distance routing methods and measuring the resulting time savings.

2.0 Problem Model

2.1 Problem Representation

The flood emergency delivery system in this study is modelled using graph theory as a weighted graph $G = (V, E)$, where the transportation network is represented in a structured mathematical form to support route optimization.

In this model, all physical locations involved in the disaster response operation are abstracted into nodes, while the road connections between these locations are represented as edges. Each edge is assigned a numerical value known as a weight, which represents the travel time required to move between two connected locations under flood-affected conditions. The weight of an edge represents the travel time (in minutes) or distance between two locations. In this project, travel time is mainly used as weight, because it can more realistically reflect the actual transportation efficiency and delay under emergency delivery conditions.

Table 2.1.1: Graph Notation and Definitions

Symbol	Description
V	The set of vertices (nodes), which include the central warehouse and all affected villages
E	The set of edges, which correspond to the road links between locations
$w(u, v)$	The weight of an edge between node u and node v , measured in travel time (minutes)

The graph consists of a total of 8 nodes, where node 0 represents the central disaster relief warehouse, and nodes 1 to 7 represent the affected villages that require emergency supply distribution.

2.2 Objective Function

The objective of the system is to determine the shortest travel time from the source node (node 0) to all other nodes in the graph.

For any path P , the total cost is defined as:

$$\text{Cost}(P) = \sum w(u, v)$$

The goal is to minimize:

$$\min \sum w(u, v)$$

This corresponds to computing the shortest path from the warehouse to all villages using Dijkstra's Shortest Path Algorithm.

2.3 Constraints

The model operates under several real-world constraints. Firstly, all edge weights are non-negative, as travel time cannot be negative in practical scenarios.

$$w(u, v) = 0$$

Secondly, not all villages are directly connected, meaning that some locations can only be reached through intermediate nodes. In addition, certain routes may become longer due to flood conditions, which affect road accessibility and increase travel time. Finally, the network may require indirect routing through intermediate villages in order to reach the destination efficiently. These constraints reflect the realistic limitations of the emergency transportation system under disaster conditions.

3.0 Algorithm Choice

When developing an emergency dispatch system for a flood-affected region such as Kuala Langat, selecting the most suitable algorithm is a crucial design decision. The system must determine the fastest routes across a network of 25 roads, many of which may be partially flooded, congested, or damaged. Since emergency supplies must reach isolated villages as quickly as possible, the chosen algorithm must not only be mathematically correct but also practical for real world disaster operations.

To identify the most appropriate solution, five major computing paradigms are evaluated based on their ability to handle route optimization within a dynamic flood environment.

3.1 Sorting

Sorting algorithms such as QuickSort, MergeSort, and HeapSort, are designed to arrange data into a specific order based on a chosen criterion. For example, MergeSort repeatedly divides a dataset into smaller sections, sorts them individually, and then combines them into a fully ordered list. This process achieves an efficient time complexity of $O(n \log n)$.

In disaster management, sorting can be useful for prioritizing relief efforts. If emergency planners receive information about population size, flood severity, or available medical supplies from all affected villages, a sorting algorithm can quickly rank villages according to urgency.

However, sorting algorithms have a major limitation, which is that they do not understand geographical relationships. They can identify which village should receive aid first, but they cannot determine how to reach that village. Concepts such as intersections, road conditions, travel times, and alternative routes fall completely outside the scope of sorting algorithms.

As a result, sorting techniques are useful for prioritization but cannot be used as the primary method for route planning and navigation.

3.2 Divide and Conquer

The Divide and Conquer (D&C) paradigm solves large problems by dividing them into smaller, independent subproblems. Each subproblem is solved separately, and the individual solutions are then combined to form a complete answer.

A common example is Binary Search, which repeatedly eliminates half of a sorted dataset until the target value is found. This approach achieves an efficient time complexity of $O(\log n)$. Divide and Conquer techniques are also commonly used in computational geometry problems, such as finding the closest pair of points.

The success of Divide and Conquer depends on one important condition: the subproblems must be independent of one another. Unfortunately, road networks in flood situations do not satisfy this requirement.

Consider dividing the Kuala Langat map into eastern and western sectors. While this may appear reasonable mathematically, routes within one sector often depend on roads located in the other sector. For example, a flooded highway in the western area may force a relief vehicle to travel through roads located in the eastern area before reaching its destination.

Because roads are interconnected and routes frequently cross regional boundaries, dividing the network into separate sections destroys the continuity needed to find valid paths. Consequently, Divide and Conquer is not suitable for solving the flood routing problem.

3.3 Dynamic Programming

Dynamic Programming (DP) is designed for optimization problems that contain overlapping subproblems. Instead of solving the same calculations repeatedly, DP stores previously computed results in memory and reuses them whenever needed. This technique significantly improves efficiency for many complex computational tasks.

A well-known example is the Floyd-Warshall Algorithm, which calculates the shortest paths between every pair of nodes in a network. By repeatedly evaluating each node as a possible intermediate point between two other nodes, it generates a complete shortest path matrix with a time complexity of $O(V^3)$.

Another example is the Bellman-Ford Algorithm, which can determine shortest paths even when negative edge weights are present.

From a mathematical perspective, Dynamic Programming can successfully solve the flood-routing problem. However, it is far more powerful than necessary for the requirements of this project.

The Floyd-Warshall algorithm calculates routes between every possible pair of locations. This means it would determine paths not only from the Shah Alam disaster hub to the villages, but also between every village to village combination. In practice, these additional calculations provide little value because all emergency supplies originate from a single location.

The problem therefore follows a single source shortest path structure rather than an all pairs shortest path structure.

Additionally, transportation networks only contain positive travel costs because moving along any road requires time and resources. As a result, advanced capabilities such as handling negative edge weights are unnecessary. Running computationally expensive algorithms on field

laptops during disaster operations would waste processing resources without improving operational outcomes.

For these reasons, Dynamic Programming is considered an overly complex solution for this scenario.

3.4 Greedy Algorithms

Efficient route planning is essential during flood emergencies because relief supplies must reach affected communities as quickly as possible while avoiding inaccessible roads. To address this problem, this project adopts the Greedy Paradigm, implemented through Dijkstra's Algorithm, to determine the fastest delivery routes from the Shah Alam warehouse to the seven flood affected villages in Kuala Langat. Dijkstra's algorithm was chosen because it is specifically designed to compute the shortest path in a weighted network by selecting the node with the smallest cumulative travel time at each step. This makes it suitable for emergency logistics, where the main objective is to minimize total travel time rather than physical distance.

The greedy paradigm was selected because it is capable of making fast and reliable decisions while supporting shortest-path computation in networks with non-negative edge weights. Since the road network in this project uses travel time as the edge weight and all values are non-negative, Dijkstra's Algorithm provides an appropriate and efficient solution for optimizing emergency logistics.

A greedy algorithm is an algorithmic design strategy that constructs a solution by repeatedly selecting the option that provides the greatest immediate benefit. Instead of evaluating every possible combination, the algorithm makes a locally optimal decision at each step with the objective of achieving the best overall solution.

Once a decision has been made, it is not reconsidered during later stages of execution. This allows greedy algorithms to produce solutions efficiently while requiring relatively little computational memory. Due to this characteristic, greedy algorithms are widely used in optimization problems such as shortest path computation, scheduling, resource allocation, and network routing.

From a computational perspective, greedy algorithms are generally efficient because they avoid exhaustive search. The time complexity of a greedy algorithm depends on the specific problem and the data structure used to select the next best option. In many cases, the greedy decision can be made in $O(1)$ or $O(\log n)$ time per iteration, resulting in an overall complexity that is often linear or near linear such as $O(n)$ to $O(n \log n)$. This makes greedy methods suitable for time sensitive applications where rapid decision making is required.

The fundamental principle behind the greedy paradigm is the relationship between local optimization and global optimization. A local optimum refers to the best available decision at the current stage of the algorithm, whereas a global optimum represents the best overall solution

after considering the complete problem. Greedy algorithms assume that repeatedly selecting locally optimal decisions will eventually produce the desired global solution.

However, this assumption is not valid for every optimization problem. In some situations, making the best immediate decision may prevent the algorithm from finding the best overall solution because earlier decisions cannot be revised.

For example, consider a delivery vehicle arriving at an intersection with two possible routes. One road initially appears faster because it has a lower travel time, while another road requires a longer initial journey. A basic greedy algorithm would immediately select the faster looking road. However, if that road later becomes inaccessible due to flooding or contains a collapsed bridge, the vehicle would be forced to turn back, resulting in a much longer overall journey. This example demonstrates that a simple greedy strategy based solely on immediate cost may not always produce the optimal route.

3.6 Justification of the Selected Algorithm

To determine the most suitable approach for the Kuala Langat emergency dispatch system, each computational paradigm was evaluated against the operational requirements of the disaster scenario.

Algorithmic Paradigm	Focus	Limitation in Flood Scenario	Decision
Sorting	Organizing data according to priority or attributes	Cannot represent road networks or determine travel paths	Rejected
Divide & Conquer	Solving independent subproblems recursively	Road networks contain overlapping and interconnected routes	Rejected
Dynamic Programming	Large scale optimization through stored computations	Excessive computational overhead for a single source routing problem	Rejected

Greedy (Dijkstra)	Continuously select the best minimum cost immediate option.	all travel time values are non negative.	SELECTED
------------------------------	---	--	-----------------

The evaluation clearly shows that graph theory provides the most appropriate framework for representing the geographical structure of the disaster area. Within graph theory, the problem can be formally classified as a Single Source Shortest Path (SSSP) problem, where all routes originate from a single distribution center and travel costs are represented by positive edge weights. This classification directly points to Dijkstra's Algorithm as the most suitable solution.

3.7 Selection of Dijkstra's Algorithm

Although a basic greedy algorithm may not always guarantee an optimal solution, Dijkstra's Algorithm overcomes this limitation by applying the greedy paradigm in a structured manner. Instead of selecting the road with the smallest immediate travel cost, Dijkstra algorithm always selects the unvisited node with the smallest cumulative travel time from the source node.

This distinction is important because every decision is based on the total travel time accumulated from the starting location rather than on individual road segments. As a result, the algorithm continuously expands the search outward while maintaining the shortest known travel time to every reachable intersection.

Whenever a shorter route to a neighboring node is discovered, the algorithm updates the recorded travel time through a process known as edge relaxation. After all neighboring nodes have been evaluated, the algorithm permanently selects the unvisited node with the smallest cumulative travel time and continues the search from that location. This process is repeated until the shortest paths to all destination villages have been determined.

By evaluating cumulative travel times instead of immediate road costs, Dijkstra's algorithm avoids many of the limitations associated with naive greedy approaches and guarantees the shortest path for graphs with non-negative edge weights.

3.7.1 Suitability for Flood Route Optimization

Dijkstra's algorithm is particularly suitable for this project because the characteristics of the flood routing problem satisfy the algorithm's assumptions.

1. Non-Negative Travel Time

Each road segment in the transportation network is assigned a travel time representing the effect of flooding, road conditions, and traffic congestion. Since travel time cannot be negative, every edge weight in the graph is non-negative. This satisfies the fundamental requirement of Dijkstra's Algorithm and guarantees that once the shortest travel time to a node has been determined, no alternative route discovered later can produce a shorter result.

2. Fast Route Computation

During emergency response operations, route planning must be completed quickly so that relief supplies can be dispatched without unnecessary delays. Dijkstra's algorithm efficiently computes the shortest routes by exploring only the most promising paths rather than evaluating every possible route combination. This enables the system to generate optimized delivery routes within a short computation time to make it suitable for time critical disaster response.

3. Reliable Route Optimization

The objective of this project is to minimize the total travel time from the Shah Alam warehouse to all seven flood-affected villages. Dijkstra's algorithm systematically evaluates the transportation network by updating shorter routes whenever they are discovered and permanently confirming the shortest path once it has been proven to be optimal. This guarantees that every recommended delivery route represents the minimum cumulative travel time under the given flood conditions.

4. Scalability

As additional villages, road intersections, or transportation routes are incorporated into the system, Dijkstra's Algorithm can continue to compute shortest paths without requiring changes to the overall routing methodology. This allows the proposed system to be expanded for larger disaster management scenarios while maintaining consistent routing performance.

Overall, Dijkstra's algorithm combines the efficiency of the greedy paradigm with mathematically guaranteed shortest path computation for networks containing non-negative edge weights. These characteristics make it an appropriate algorithm for optimizing emergency clothing distribution routes during flood disasters in Kuala Langat.

4.0 Algorithm Design

4.1 Overview of Dijkstra's Algorithm

Dijkstra's algorithm is one of the most well-known and widely used algorithms in computer science, specifically designed to solve the Single-Source Shortest Path (SSSP) problem. It was conceived by Dutch computer scientist Edsger W. Dijkstra in 1956 and published in 1959. The algorithm determines the shortest path from a designated starting node to every other node in a weighted graph, where all edge weights must be non-negative.

In the context of this project, Dijkstra's algorithm is applied to a flood disaster emergency delivery network, where:

- The source node is the Warehouse (node 0)
- The destination nodes are seven flood-affected villages (nodes 1–7)
- The edge weights represent estimated travel time in minutes along each road

The algorithm guarantees that once a node is finalized (marked as visited), the shortest path to that node has been found and will not change. This property, known as the greedy property, is what makes Dijkstra's algorithm both efficient and reliable for this type of problem.

4.2 Step-by-step working principle

Dijkstra's algorithm operates through the following key steps:

Step 1 — Initialization

All vertices in the graph are assigned an initial travel time of infinity (∞), except the source vertex (Warehouse), which is set to 0. A priority queue is used to manage vertices based on their current known shortest time.

Step 2 — Node Selection

The vertex with the smallest known travel time is removed from the priority queue and becomes the current vertex. This vertex is then marked as visited, meaning its shortest travel time from the source has been confirmed.

Step 3 — Edge Relaxation

For each vertex directly connected to the current vertex (adjacent vertex), the algorithm calculates a new possible travel time from the source through the current vertex:

$$\text{newDist} = \text{dist}[\text{current}] + \text{edge.weight}$$

This represents the total travel time from the source $\rightarrow$ current vertex $\rightarrow$ adjacent vertex.

If newDist is smaller than the previously recorded travel time for that adjacent vertex, the travel time is updated (relaxed), and the vertex is inserted into the priority queue with the updated travel time.

Step 4 — Repetition

Steps 2 and 3 are repeated until the priority queue is empty, meaning all reachable vertices have been processed and their shortest travel time finalized.

Step 5 — Result

The `dist[]` array stores the shortest travel time from the source vertex (Warehouse) to every other vertex in the graph.

4.3 Pseudocode

Dijkstra(Graph, Start)

1. For each vertex `v` in Graph:

`distance[v] = infinity`

`visited[v] = false`

2. `distance[Start] = 0`

3. Create a priority queue PQ

 Insert (Start, 0) into PQ

4. While PQ is not empty:

`current = vertex removed from PQ with shortest travel time`

 If `visited[current] == true`:

`continue`

 Mark `visited[current] = true`

 For each neighbour `n` of `current`:

`newDistance = distance[current] + edge weight(current, n)`

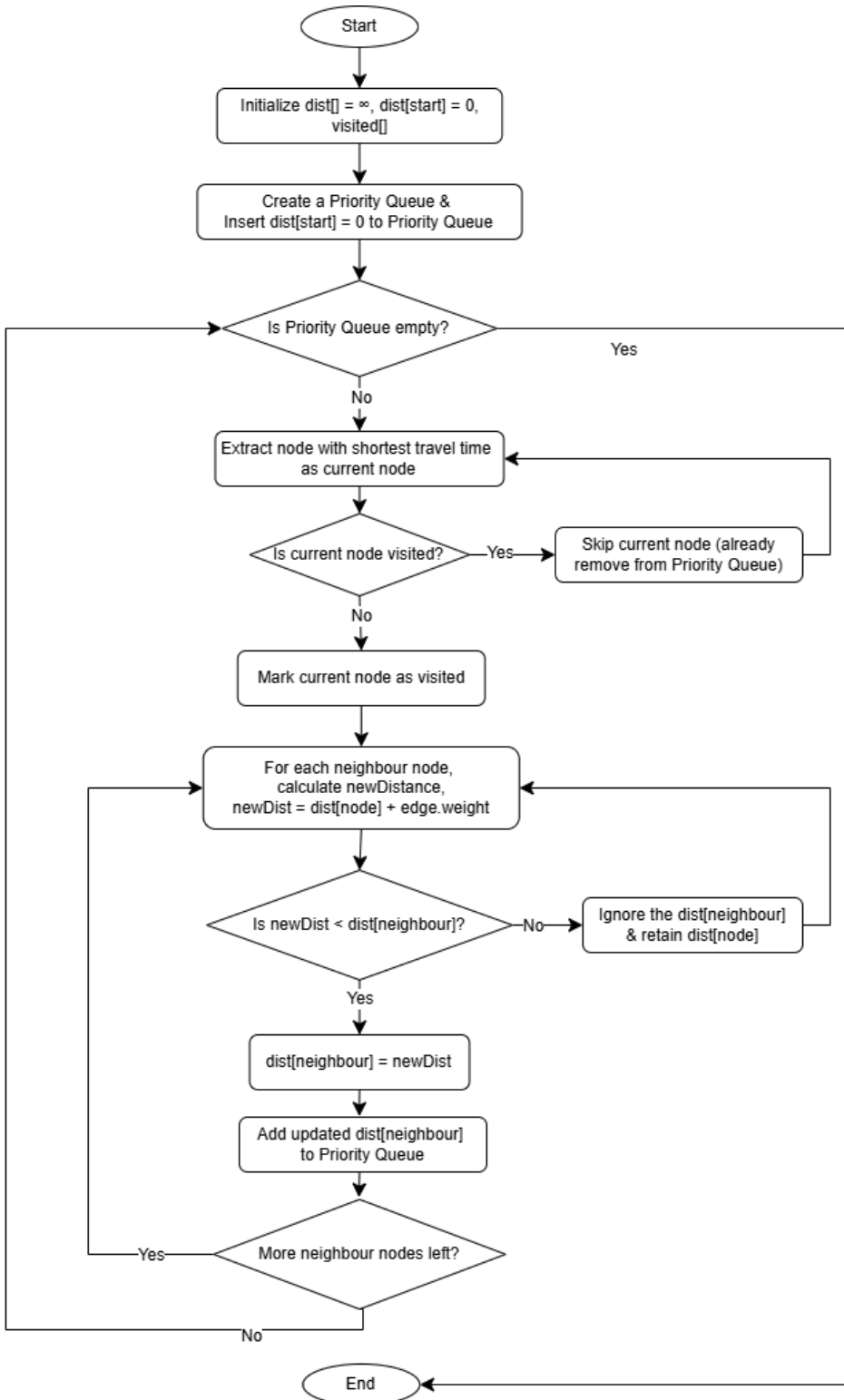
 If `newDistance < distance[n]`:

`distance[n] = newDistance`

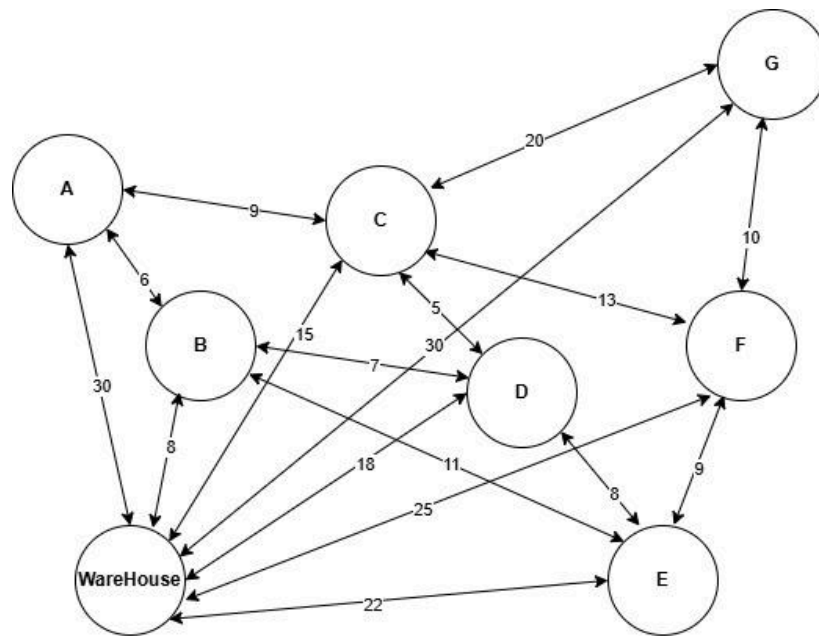
 Insert (`n`, `newDistance`) into PQ

5. Return `distance[]`

4.4 Flowchart



4.5 Original Graph Diagram



5.0 Complexity Analysis

The time complexity of Dijkstra's algorithm depends on the underlying data structure used to manage the priority of unvisited nodes. In this implementation, a min-heap-based priority queue is employed, which significantly improves efficiency over a naive array-based approach.

The algorithm processes each vertex V exactly once upon extraction from the priority queue, and each extraction operation costs $O(\log V)$ time. For every edge E in the graph, a relaxation step is performed, which may involve updating the priority queue at a cost of $O(\log V)$ per operation. Combining both operations, the overall time complexity is expressed as:

$$T(V, E) = O((V + E) \log V)$$

For a connected graph where $E \geq V$, this simplifies to:

$$T(V, E) = O(E \log V)$$

Table 5.1.1: Time Complexity across All Cases

Case	Graph Structure	When It Happens	Time Complexity
Best Case	Highly Sparse ($E \approx V$)	Nodes form a chain or tree with minimal alternative paths	$O(V \log V)$
Average Case	Normal Network ($V < E < V^2$)	Real-world road networks or IoT routing systems	$O((V+E) \log V)$
Worst Case	Highly Dense ($E \approx V^2$)	Every node is connected to almost every other node	$O(V^2 \log V)$

In the best-case scenario, the graph is sparse, meaning each node has very few neighbors. This reduces the number of edge relaxations, resulting in approximately $O(V \log V)$ performance dominated by heap extraction operations.

In the average case, which represents most real-world applications (such as road maps or network routing systems), the algorithm processes a moderate number of edges, resulting in a balanced complexity of $O((V+E) \log V)$.

In the worst-case scenario, the graph becomes extremely dense, where every vertex is connected to almost all other vertices. In this case, $E \approx V^2$, and the complexity expands to $O(V^2 \log V)$ due to frequent edge relaxations and priority queue updates.

Although the graph structure affects the number of operations, the presence of the binary heap ensures that all priority queue operations remain $O(\log V)$. This is what allows Dijkstra's algorithm to scale efficiently compared to a naive implementation, which would require $O(V^2)$ time due to linear search for the minimum distance node.

6.0 Implementation

6.1 Overview

The system is implemented in Java and consists of two main components: a graph representation using an adjacency list (List<List<Edge>>), and the core Dijkstra's algorithm function that computes the shortest delivery time from the Warehouse to all seven villages. The Edge class is a simple data structure storing the destination node (to) and the travel time (weight) of a road segment.

6.2 Full Source Code

```
1 package mypackage;
2
3 import java.util.*;
4
5
6 class Edge {
7     int to;
8     int weight;
9
10
11 Edge(int to, int weight) {
12     this.to = to;
13     this.weight = weight;
14 }
15 }
16
17
18 public class FloodGraphSystem {
19
20
21     static int V = 8;
22     // 0 = Warehouse
23     // 1 = A, 2 = B, 3 = C, 4 = D, 5 = E, 6 = F, 7 = G
24
25
26     static String[] name = {
27         "Warehouse",
28         "Village A",
29         "Village B",
30         "Village C",
31         "Village D",
32         "Village E",
33         "Village F",
34         "Village G"
35     };
36
37 public static void buildPath(int[] parent, int node, StringBuilder path) {
38     if (node == -1) return;
39
40
41     if (parent[node] != -1) {
42         buildPath(parent, parent[node], path);
43         path.append(" -> ");
44     }
45
46     path.append(node == 0 ? "Warehouse" : name[node]);
47 }
48
49 }
```

```

50 public static void dijkstra(List<List<Edge>> graph, int start) {
51
52     // dist[i] = shortest known distance from start to node i
53     int[] dist = new int[V];
54
55     // visited[i] = true once node i's shortest path is finalized; never updated again
56     boolean[] visited = new boolean[V];
57
58     // parent[i] = predecessor of node i on the shortest path from start
59     int [] parent = new int[V];
60
61     Arrays.fill(dist, Integer.MAX_VALUE);
62     Arrays.fill(parent, -1);
63
64     dist[start] = 0;
65
66     // Min-heap ordered by dist (2nd element) - top is always the current smallest (node, dist)
67     PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
68
69     pq.add(new int[]{start, 0});
70
71     while (!pq.isEmpty()) {
72
73         int[] current = pq.poll();
74         int node = current[0];
75
76         // The heap may still hold stale entries for this node (older, larger dist)
77         // Once a node is finalized, skip it instead of re-expanding its neighbors
78         if (visited[node]) continue;
79         visited[node] = true;
80
81
82         for (Edge edge : graph.get(node)) {
83
84             // Relaxation: check if going through this node gives a shorter path than the current best known distance
85             if (dist[node] + edge.weight < dist[edge.to]) {
86                 dist[edge.to] = dist[node] + edge.weight;
87                 parent[edge.to] = node;
88                 // Old (edge.to, oldDist) entry is left in the heap, not removed -
89                 // it gets filtered out later by the visited check above
90                 pq.add(new int[]{edge.to, dist[edge.to]});
91             }
92         }
93     }
94
95     // =====
96     // CLEAN OUTPUT (3 COLUMNS ONLY)
97     // =====
98
99     System.out.println("\n===== SHORTEST PATH RESULT =====\n");
100
101     System.out.printf("%-12s %-40s %-15s\n",
102         "Village", "Shortest Path", "Time Taken (min)");
103
104     System.out.println("-----");
105
106     for (int i = 0; i < V; i++) {
107
108         StringBuilder path = new StringBuilder();
109         buildPath(parent, i, path);
110
111         System.out.printf("%-12s %-40s %-15d\n",
112             name[i],
113             path.toString(),
114             dist[i]);
115     }
116
117     System.out.println("\n");
118 }
119
120 public static void main(String[] args) {
121
122     List<List<Edge>> graph = new ArrayList<>();
123
124     for (int i = 0; i < V; i++) {
125         graph.add(new ArrayList<>());
126     }
127
128 }

```

```

133 // Flood Disaster Road Network (7 Villages)
134 // Undirected graph: each road is added twice (forward + reverse)
135
136 // =====
137 // Warehouse → Villages
138 // =====
139
140 graph.get(0).add(new Edge(1, 30));
141 graph.get(1).add(new Edge(0, 30));
142
143 graph.get(0).add(new Edge(2, 8));
144 graph.get(2).add(new Edge(0, 8));
145
146 graph.get(0).add(new Edge(3, 15));
147 graph.get(3).add(new Edge(0, 15));
148
149 graph.get(0).add(new Edge(4, 18));
150 graph.get(4).add(new Edge(0, 18));
151
152 graph.get(0).add(new Edge(5, 22));
153 graph.get(5).add(new Edge(0, 22));
154
155 graph.get(0).add(new Edge(6, 25));
156 graph.get(6).add(new Edge(0, 25));
157
158 graph.get(0).add(new Edge(7, 30));
159 graph.get(7).add(new Edge(0, 30));
160
161 // =====
162 // Village ↔ Village
163 // =====
164
165 // A (1)
166 graph.get(1).add(new Edge(2, 6));
167 graph.get(2).add(new Edge(1, 6));
168
169 graph.get(1).add(new Edge(3, 9));
170 graph.get(3).add(new Edge(1, 9));
171
172 // B (2)
173 graph.get(2).add(new Edge(4, 7));
174 graph.get(4).add(new Edge(2, 7));
175
176 graph.get(2).add(new Edge(5, 11));
177 graph.get(5).add(new Edge(2, 11));
178
179 // C (3)
180 graph.get(3).add(new Edge(4, 5));
181 graph.get(4).add(new Edge(3, 5));
182
183 graph.get(3).add(new Edge(6, 13));
184 graph.get(6).add(new Edge(3, 13));
185
186 graph.get(3).add(new Edge(7, 20));
187 graph.get(7).add(new Edge(3, 20));
188
189 // D (4)
190 graph.get(4).add(new Edge(5, 8));
191 graph.get(5).add(new Edge(4, 8));
192
193 // E (5)
194 graph.get(5).add(new Edge(6, 9));
195 graph.get(6).add(new Edge(5, 9));
196
197 // F (6)
198 graph.get(6).add(new Edge(7, 10));
199 graph.get(7).add(new Edge(6, 10));
200
201 // Run Dijkstra
202 dijkstra(graph, 0);
203 }
204 }
205 }
206 }
207 }
208 }
209 }

```

6.3 Explanation of Key Parts

Graph representation

The road network is represented using an adjacency list, `List<List<Edge>>` graph, where `graph.get(i)` returns all roads directly connected to node `i`. Since the flood emergency road network is undirected, meaning vehicles can travel in both directions along a road, each connection is added twice: once from the source node to the destination node and once in the opposite direction. This ensures that bidirectional travel is correctly represented.

Distance array (dist[])

This array stores the shortest known travel time from the Warehouse to every node. In Java, `Integer.MAX_VALUE` is the largest value representable by the `int` data type (2,147,483,647), and it is used here as a practical substitute for infinity, since it is guaranteed to be larger than any real distance value in the graph. Initially, all distances are set to `Integer.MAX_VALUE` to represent that no path has been found yet, except for the starting node, whose distance is set to 0 because the Warehouse is already at its own location.

Visited array (visited[])

This array keeps track of nodes whose shortest distances have been finalized. It is important for the correctness of the algorithm because the priority queue may contain multiple entries for the same node that were added before a shorter path was found. Once a node has been finalized, the visited array prevents it from being processed again.

Parent array (parent[])

This array stores the predecessor of each node along the shortest path from the Warehouse. Initially, all values are set to -1, indicating that no predecessor has been assigned. During the relaxation step, whenever a shorter path to a neighbour is found, `parent[edge.to]` is updated to the current node. After the algorithm finishes, the parent array is used in the `buildPath()` method to trace back the predecessors and reconstruct the full shortest path from the Warehouse to each village for display purposes.

Priority queue (pq)

The priority queue is implemented internally as a min-heap, a binary tree structure in which the value of each parent node is less than or equal to the values of its children. This property ensures that the smallest element in the queue is always located at the root of the tree. The min-heap is maintained through two main operations: insertion and extraction.

When a new entry is added using `pq.add()`, it is first placed at the next available position at the bottom of the heap. It is then repeatedly compared with its parent and swapped upward, a process known as sift-up, until the min-heap property is restored.

When `pq.poll()` is called, the root element, which contains the smallest distance currently stored in the queue, is removed and returned. The last element in the heap is then moved to the root position and repeatedly compared with its children, swapping downward through a process called sift-down until the min-heap property is satisfied again.

Because of this structure, `pq.poll()` always retrieves the unvisited node with the smallest known distance. This enables Dijkstra's algorithm to process nodes in the correct order, ensuring that the shortest paths are found efficiently.

Relaxation step

For each neighbouring edge of the current node, the algorithm checks whether travelling through the current node provides a shorter path to the neighbour than the distance currently recorded. If a shorter path is found ($\text{dist}[\text{node}] + \text{edge.weight} < \text{dist}[\text{edge.to}]$), the neighbour's distance is updated and a new entry is added to the priority queue. Older entries with longer distances are not removed immediately. Instead, they are ignored later by checking the visited array when they are removed from the priority queue.

7.0 Correctness Analysis

7.1 Sample Input

The graph used to test the system is the flood disaster road network with one Warehouse and seven villages (A–G), connected as follows:

The graph used in this system models a flood disaster emergency delivery network consisting of 8 nodes, including one Warehouse (Node 0) as the source and seven flood-affected villages (Nodes 1–7). The graph is undirected and weighted, where each edge weight represents the estimated travel time in minutes under flood-affected road conditions.

The following tables summarise the edges used as input to the algorithm.

Direct edges from Warehouse to Villages:

Village 1	Village 2	Weight (minutes)
Warehouse	Village A	30
	Village B	8
	Village C	15
	Village D	18
	Village E	22
	Village F	25
	Village G	30

Inter-village edges:

Village 1	Village 2	Weight (minutes)
Village A	Village B	6
Village A	Village C	9
Village B	Village D	7
Village B	Village E	11
Village C	Village D	5
Village C	Village F	13
Village C	Village G	20
Village D	Village E	8

Village E	Village F	9
Village F	Village G	10

7.2 Actual Program Output

Upon executing `dijkstra(graph, 0)` with the source set to Node 0 (Warehouse), the program produces the following output:

```
===== SHORTEST PATH RESULT =====
```

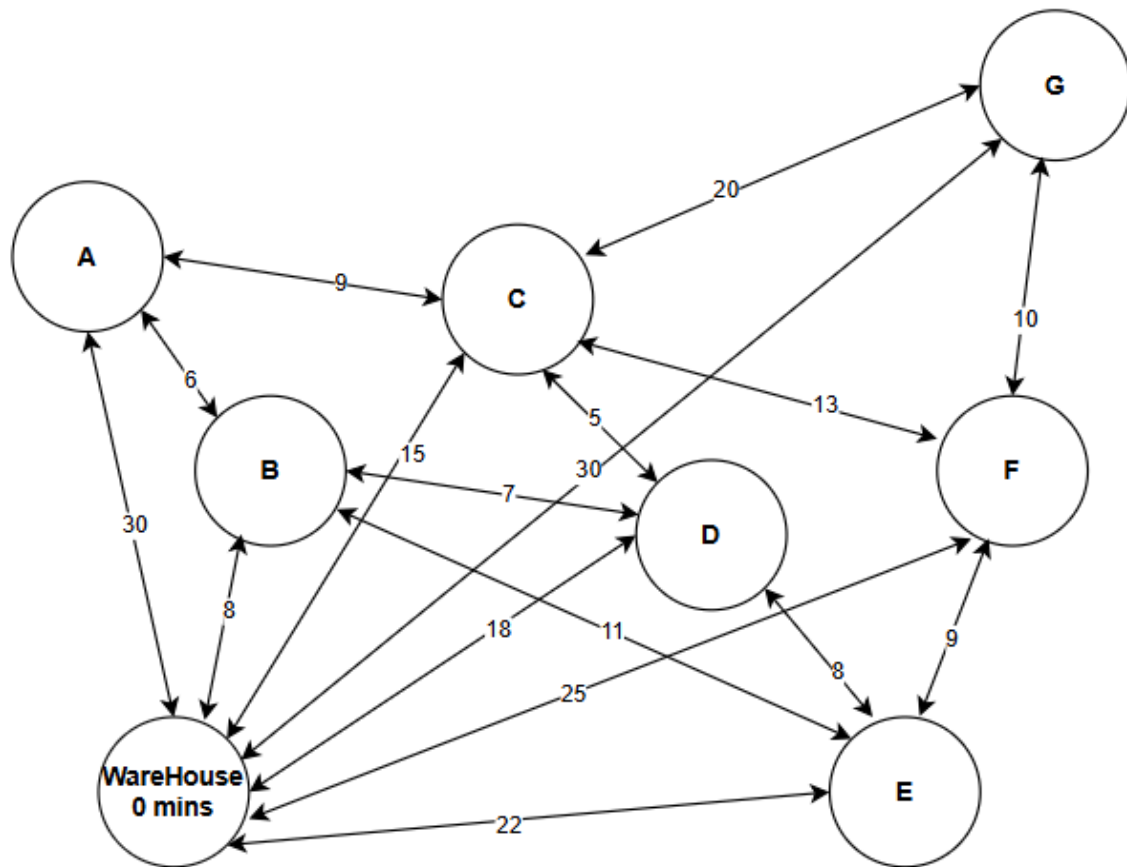
Village	Shortest Path	Time Taken (min)
Warehouse	Warehouse	0
Village A	Warehouse -> Village B -> Village A	14
Village B	Warehouse -> Village B	8
Village C	Warehouse -> Village C	15
Village D	Warehouse -> Village B -> Village D	15
Village E	Warehouse -> Village B -> Village E	19
Village F	Warehouse -> Village F	25
Village G	Warehouse -> Village G	30

7.3 Manual Verification

To validate the correctness of the program output, three destination villages are manually verified by enumerating all feasible paths from the Warehouse and comparing their total travel times against the algorithm's result.

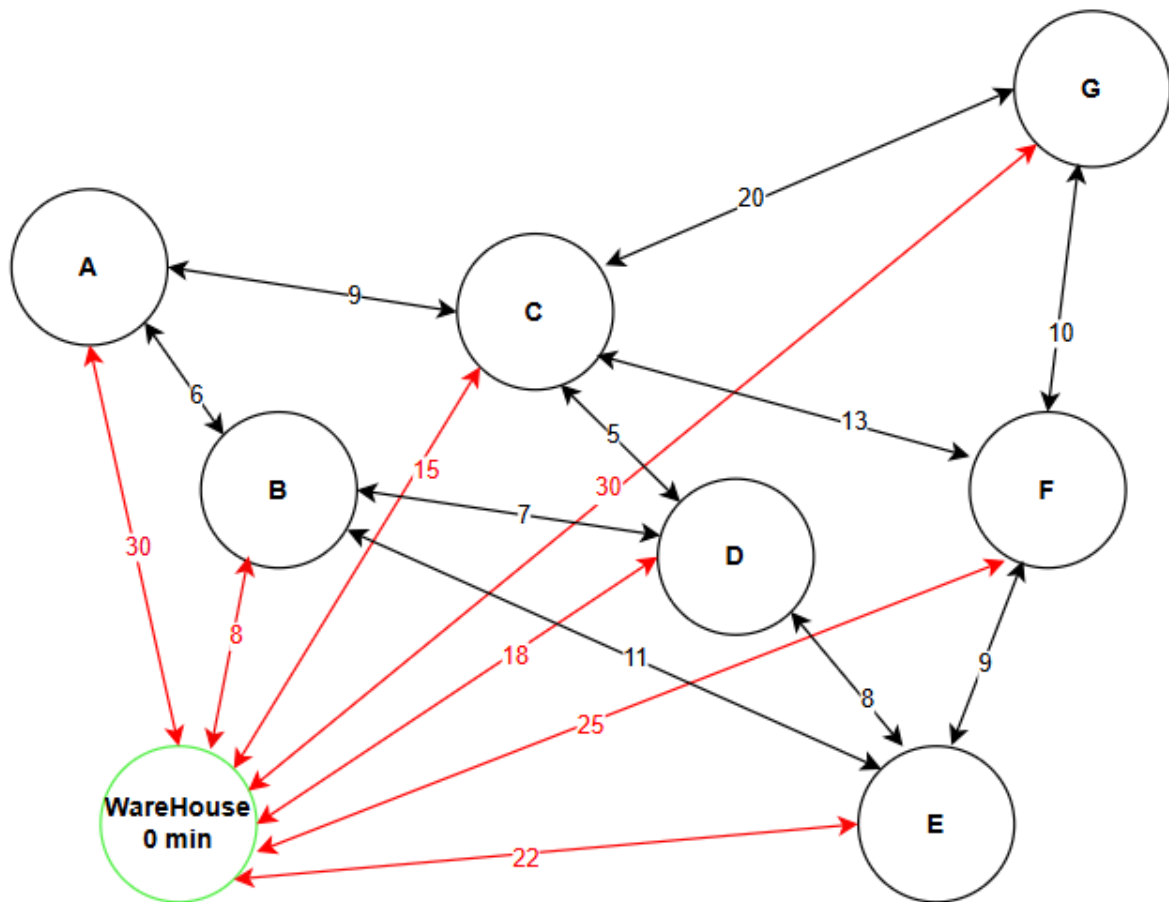
Initialization

Set `dist[Warehouse] = 0`, all others = ∞ . Add Warehouse to priority queue.



Node	Warehouse	A	B	C	D	E	F	G
dist	0	∞	∞	∞	∞	∞	∞	∞

Step 1 — Visit Warehouse (dist = 0)



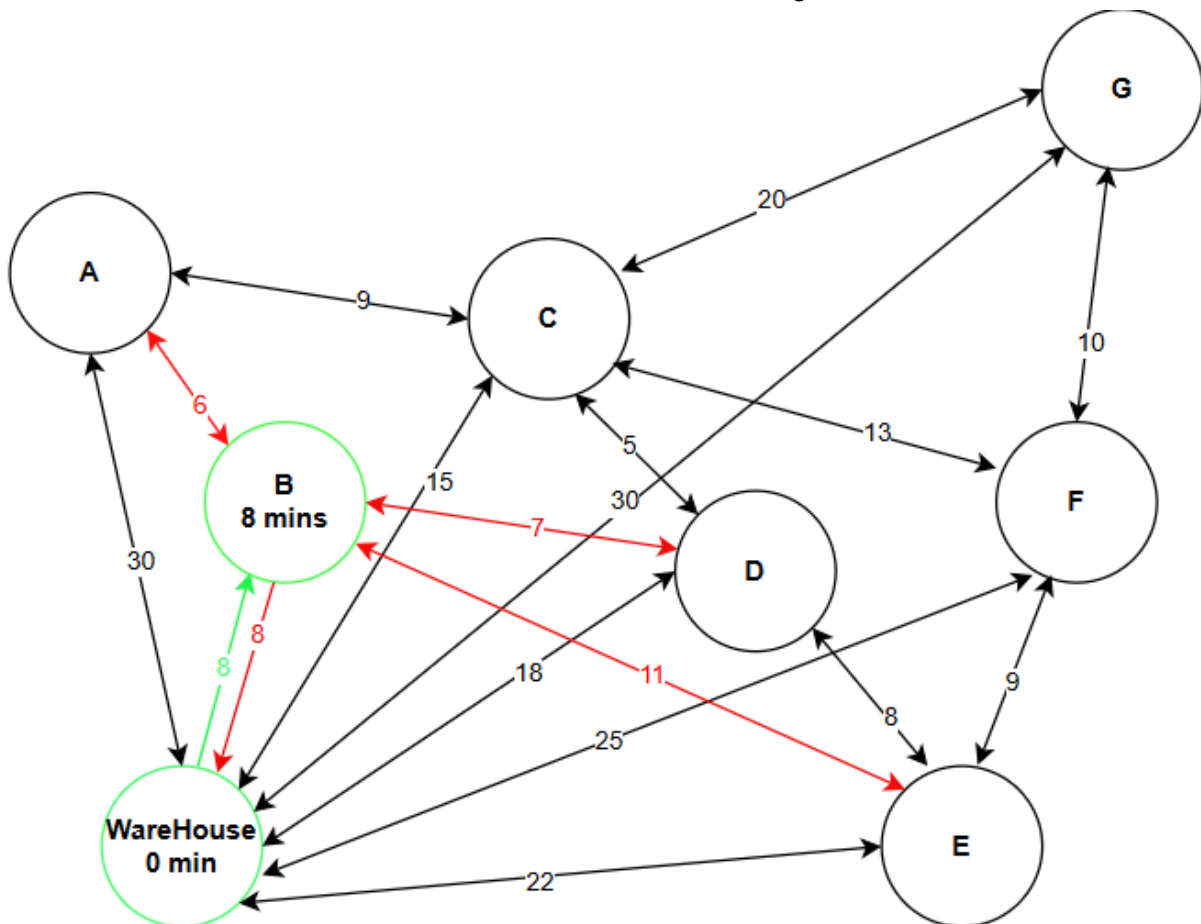
Relax all direct neighbours of Warehouse:

Neighbour	Old dist	Edge Weight	New dist	Updated?
A	∞	30	30	Yes ($30 < \infty$)
B	∞	8	8	Yes ($8 < \infty$)
C	∞	15	15	Yes ($15 < \infty$)
D	∞	18	18	Yes ($18 < \infty$)
E	∞	22	22	Yes ($22 < \infty$)
F	∞	25	25	Yes ($25 < \infty$)
G	∞	30	30	Yes ($30 < \infty$)

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	30	8	15	18	22	25	30

Step 2 — Visit B (dist = 8)

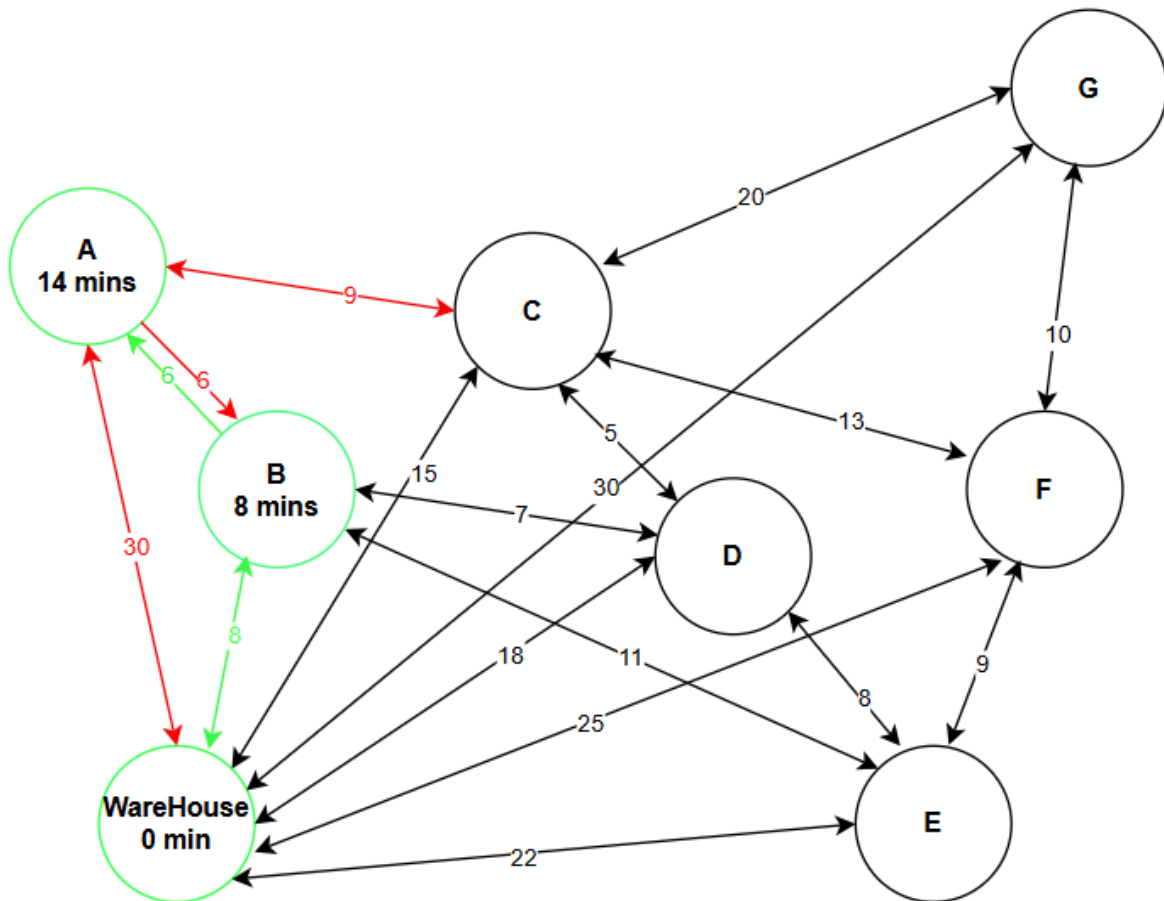
Since Village B has the smallest known distance among all unvisited nodes (dist = 8), it is selected for processing. All neighbouring nodes of Village B are then relaxed by comparing the current known distance with the new candidate distance via Village B.



Neighbour	Old dist	Calculation	New dist	Updated?
Warehouse	0	$8+8=16$	0	No ($16 > 0$)
A	30	$8+6=14$	14	Yes ($14 < 30$)
D	18	$8+7=15$	15	Yes ($15 < 18$)
E	22	$8+11=19$	19	Yes ($19 < 22$)

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14	8 ✓	15	15	19	25	30

Step 3 — Visit A (dist = 14) ← smallest unvisited



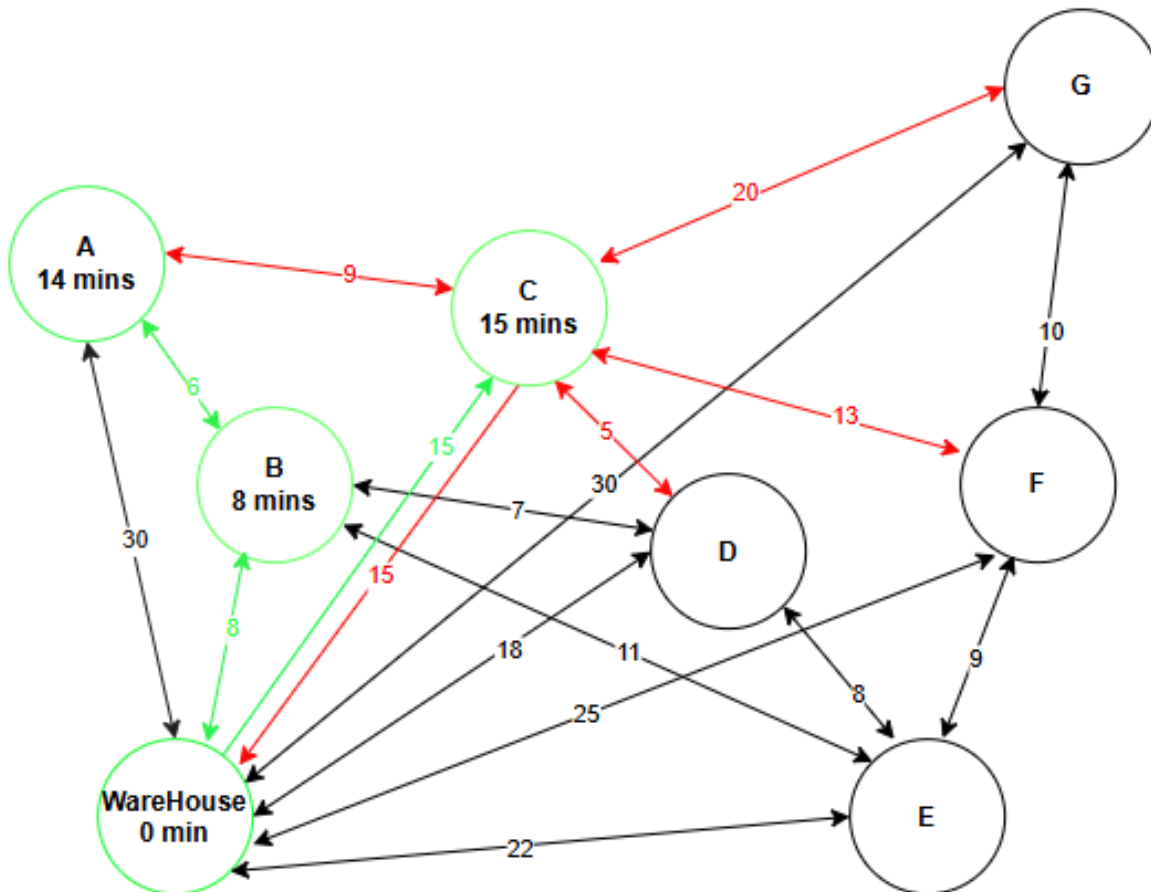
Relax all neighbours of A:

Neighbour	Old dist	Calculation	New dist	Updated?
Warehouse	0	$14 + 30 = 44$	0	No ($44 > 0$)
B	8	$14 + 6 = 20$	8	No ($20 > 8$)
C	15	$14 + 9 = 23$	15	No ($23 > 15$)

No updates.

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14 ✓	8 ✓	15	15	19	25	30

Step 4 — Visit C (dist = 15) ← smallest unvisited (tie with D, C chosen first)



Relax all neighbours of C:

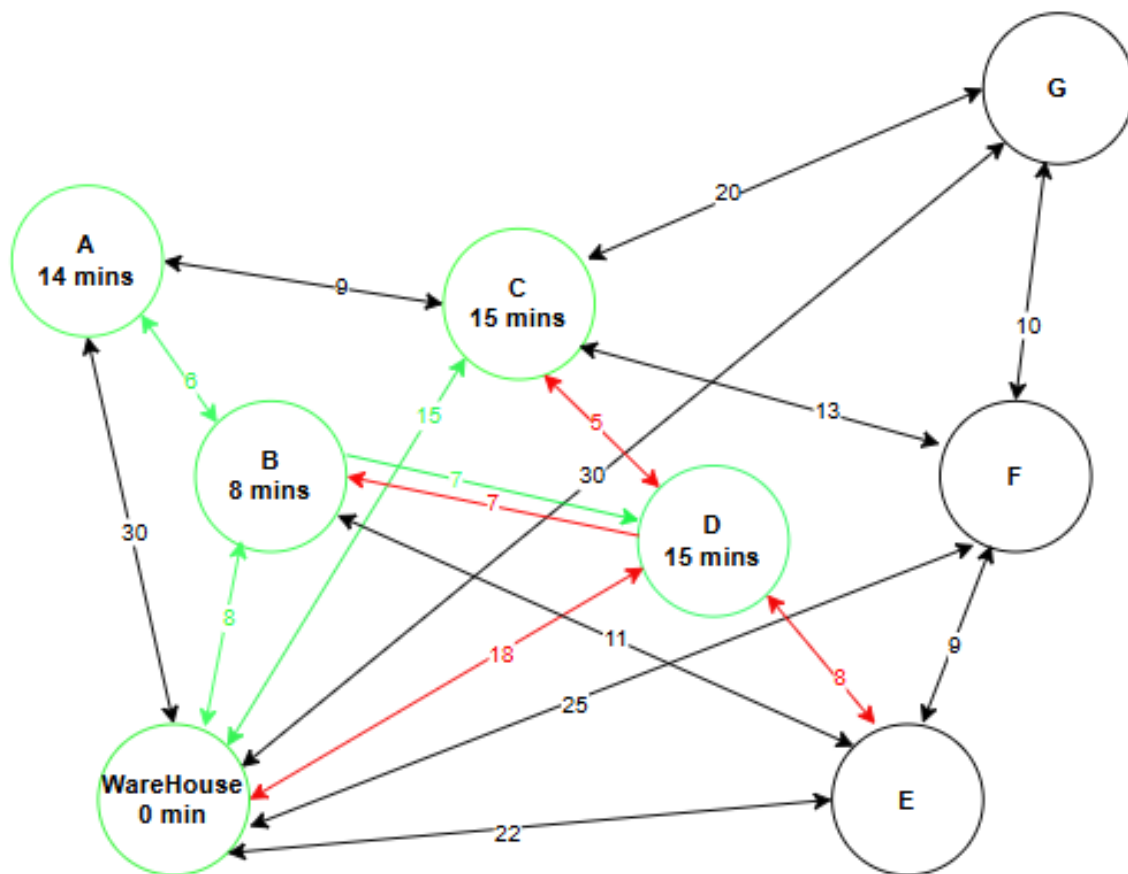
Neighbour	Old dist	Calculation	New dist	Updated?
Warehouse	0	$15 + 15 = 30$	0	No ($30 > 0$)
A	14	$15 + 9 = 24$	14	No ($24 > 14$)
D	15	$15 + 5 = 20$	15	No ($20 > 15$)
F	25	$15 + 13 = 28$	25	No ($28 > 25$)

G	30	$15+20=35$	30	No ($35 > 30$)
---	----	------------	----	------------------

No updates.

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14 ✓	8 ✓	15 ✓	15	19	25	30

Step 5 — Visit D (dist = 15) ← smallest unvisited



Relax all neighbours of D:

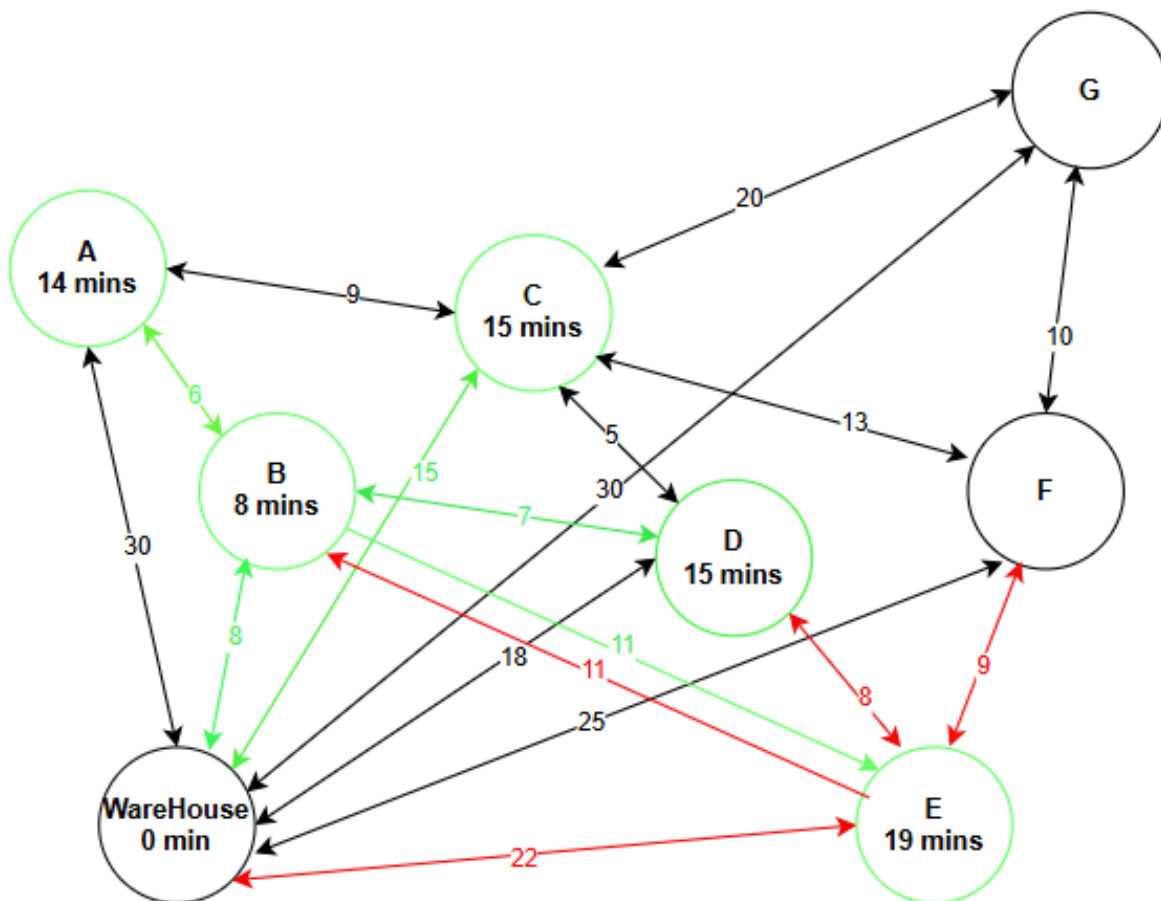
Neighbour	Old dist	Calculation	New dist	Updated?
Warehouse	0	$15+18=33$	0	No ($33 > 0$)
B	8	$15+7=22$	8	No ($22 > 8$)
C	15	$15+5=20$	15	No ($20 > 15$)

E	19	$15+8=23$	19	No ($23 > 19$)
---	----	-----------	----	------------------

No updates.

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14 ✓	8 ✓	15 ✓	15 ✓	19	25	30

Step 6 — Visit E (dist = 19) ← smallest unvisited



Relax all neighbours of E:

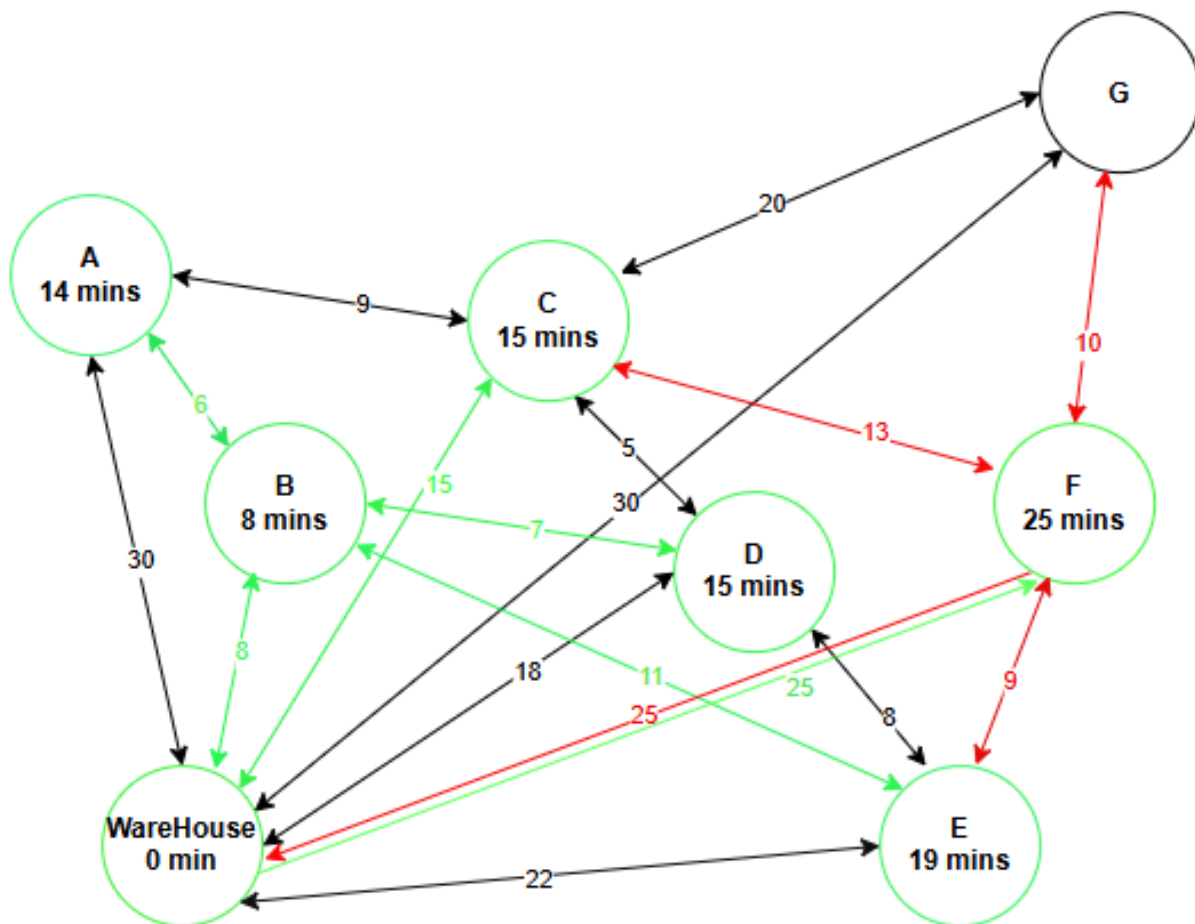
Neighbour	Old dist	Calculation	New dist	Updated?
Warehouse	0	$19+22=41$	0	No ($41 > 0$)
B	8	$19+11=30$	8	No ($30 > 8$)

D	15	$19+8=27$	15	No ($27 > 15$)
F	25	$19+9=28$	25	No ($28 > 25$)

No updates.

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14 ✓	8 ✓	15 ✓	15 ✓	19 ✓	25	30

Step 7 — Visit F (dist = 25) ← smallest unvisited



Relax all neighbours of F:

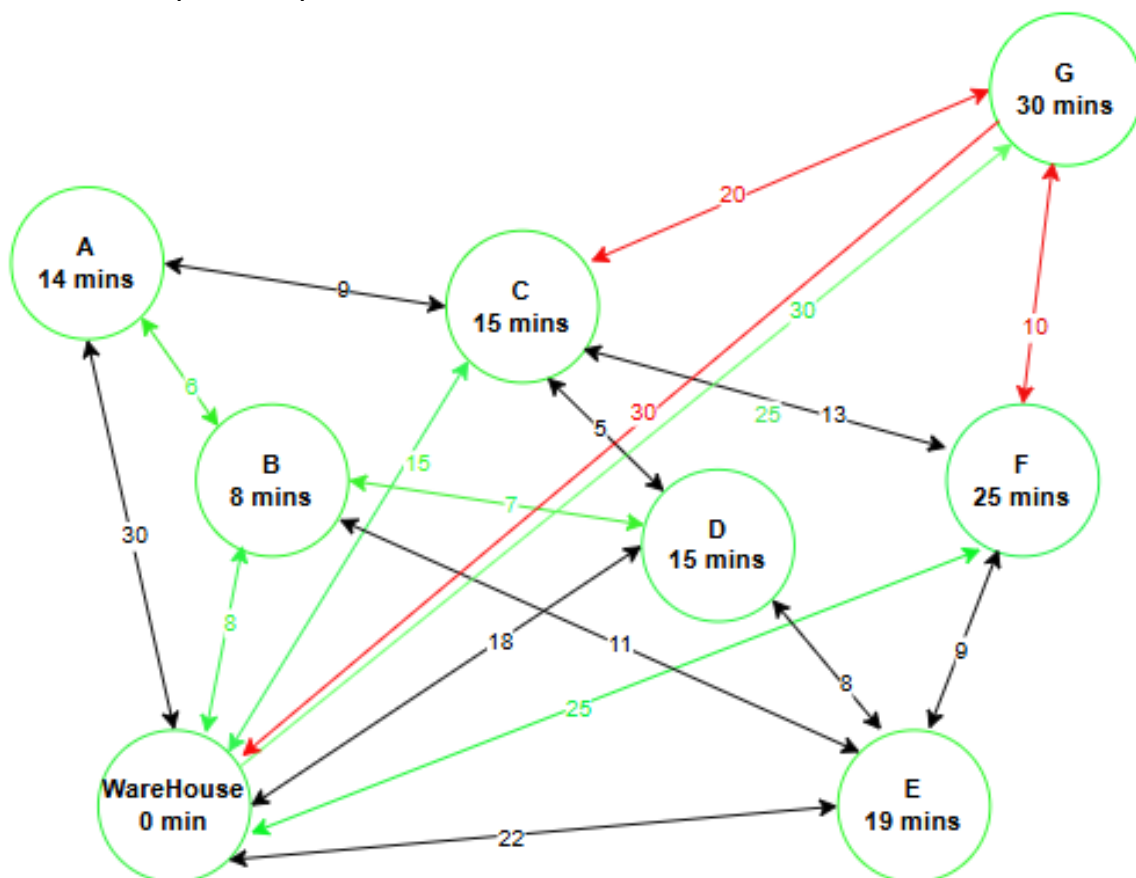
Neighbour	Old dist	Calculation	New dist	Updated?
-----------	----------	-------------	----------	----------

Warehouse	0	$25+25=50$	0	No ($50 > 0$)
C	15	$25+13=38$	15	No ($38 > 15$)
E	19	$25+9=34$	19	No ($34 > 19$)
G	30	$25+10=35$	30	No ($35 > 30$)

No updates.

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14 ✓	8 ✓	15 ✓	15 ✓	19 ✓	25 ✓	30

Step 8 — Visit G (dist = 30) ← last unvisited



Relax all neighbours of G:

Neighbour	Old dist	Calculation	New dist	Updated?
-----------	----------	-------------	----------	----------

Warehouse	0	$30+30=60$	0	No ($60 > 0$)
C	15	$30+20=50$	15	No ($50 > 15$)
F	25	$30+10=40$	25	No ($40 > 25$)

No updates.

Node	Warehouse	A	B	C	D	E	F	G
dist	0 ✓	14 ✓	8 ✓	15 ✓	15 ✓	19 ✓	25 ✓	30 ✓

Final Verified Result

Village	Shortest Path	Manual dist	Program Output	Match?
Warehouse	—	0 min	0 min	Yes
Village A	WH → B → A	14 min	14 min	Yes
Village B	WH → B	8 min	8 min	Yes
Village C	WH → C	15 min	15 min	Yes
Village D	WH → B → D	15 min	15 min	Yes
Village E	WH → B → E	19 min	19 min	Yes
Village F	WH → F	25 min	25 min	Yes
Village G	WH → G	30 min	30 min	Yes

All seven villages have been manually verified, and all results are consistent with the program output, confirming that the algorithm is producing correct shortest distances for this flood emergency delivery network.

7.4 Proof of Correctness

The correctness of Dijkstra's algorithm is formally established through two complementary arguments: the Greedy Choice Property, which justifies the algorithm's selection strategy at each step, and the Loop Invariant, which proves that the algorithm maintains and eventually produces correct shortest-path values throughout its execution.

7.4.1 Greedy Choice Property

At each iteration, Dijkstra's algorithm selects the unvisited node with the smallest current known distance $\text{dist}[\text{node}]$ from the source. This greedy strategy is valid only when all edge weights are non-negative.

The key idea is that once a node is selected and marked as visited, its shortest distance is final and cannot be improved later. This is because any alternative path to this node would have to go through another node that is either already visited or still unvisited. However, all unvisited nodes have a distance that is greater than or equal to the selected node's distance, and all edge weights are non-negative. Therefore, any alternative route cannot produce a shorter path.

In this flood emergency system, all road travel times are represented using positive integers. As a result, the non-negative weight condition is satisfied, and the greedy selection in Dijkstra's algorithm always produces an optimal result.

7.4.2 Loop Invariant

A loop invariant is a condition that remains true before and after every iteration of a loop. It is used to prove that an algorithm is correct by showing that the condition holds throughout its execution.

In Dijkstra's algorithm, the loop invariant is defined as follows:

At the beginning of each iteration of the main while-loop, for every node that has already been marked as visited, $\text{dist}[\text{node}]$ represents the true shortest distance from the source node (Warehouse) to that node.

The proof of this invariant is divided into three stages:

Initialization

Before the algorithm starts, all nodes are unvisited, meaning all values in the `visited[]` array are set to false. This indicates that no node has been finalized.

The `dist[]` array is initialised such that $\text{dist}[\text{start}] = 0$, representing the distance from the Warehouse to itself, while all other nodes are set to infinity (`Integer.MAX_VALUE`) to indicate that their shortest paths are not yet known.

At this stage, since no node has been marked as visited, the loop invariant holds trivially.

Maintenance

At each iteration, the algorithm extracts the node 'node' with the smallest tentative distance $\text{dist}[\text{node}]$ from the priority queue. If the node has already been visited, it is skipped. Otherwise, it is marked as visited (finalised).

Because all edge weights are non-negative, once a node is selected as the minimum from the priority queue, there cannot exist any alternative path through unvisited nodes that can produce

a shorter distance to that node. Therefore, $\text{dist}[\text{node}]$ is guaranteed to be the true shortest distance from the Warehouse to that node at the moment it is finalised.

After that, the algorithm performs edge relaxation on all neighbouring nodes edge.to of the current node. For each neighbour, if $\text{dist}[\text{node}] + \text{edge.weight} < \text{dist}[\text{edge.to}]$, then $\text{dist}[\text{edge.to}]$ is updated with this smaller value, and the updated node is inserted into the priority queue.

Any outdated entries in the priority queue are ignored because the algorithm checks the $\text{visited}[]$ array before processing a node. This ensures that only the first finalised shortest distance is accepted.

As a result, after each iteration, the loop invariant continues to hold for all nodes that have been marked as visited.

Termination

The algorithm terminates when the priority queue becomes empty, meaning there are no more nodes left to process. At this point, all reachable nodes have been marked as visited and their shortest distances have been finalized.

By the loop invariant, for every node in the graph, once it is marked as visited, $\text{dist}[\text{node}]$ represents the true shortest distance from the Warehouse to that node. Therefore, when the algorithm ends, every reachable node has its correct shortest distance stored in the $\text{dist}[]$ array.

Hence, the algorithm correctly computes the shortest delivery time from the Warehouse to all reachable villages.

7.5 Conclusion of Correctness

Based on the manual verification and formal proof presented above, the implementation of Dijkstra's algorithm in this flood emergency delivery system is confirmed to be correct. The algorithm satisfies all necessary conditions for correctness: all edge weights are non-negative, the graph is fully connected ensuring all villages are reachable, and the greedy selection combined with edge relaxation guarantees that each finalised distance is optimal. The program output is consistent with manually computed results across all verified cases, further supporting the validity of the implementation.

9.0 Conclusion

In conclusion, the Dijkstra algorithm plays a crucial role in the proposed Flood Emergency Delivery System by determining the shortest travel time between affected locations and available resources. By using travel time as the edge weight, the algorithm ensures that emergency supplies can be delivered through the fastest possible routes, which is essential during flood situations where timely assistance can save lives.

The implementation of a priority queue enables the algorithm to efficiently process and update routes, while the visited node mechanism prevents redundant computations and improves overall system performance. As road conditions and accessibility may change during flooding, the algorithm can continuously evaluate available paths and select the most efficient route for emergency deliveries.

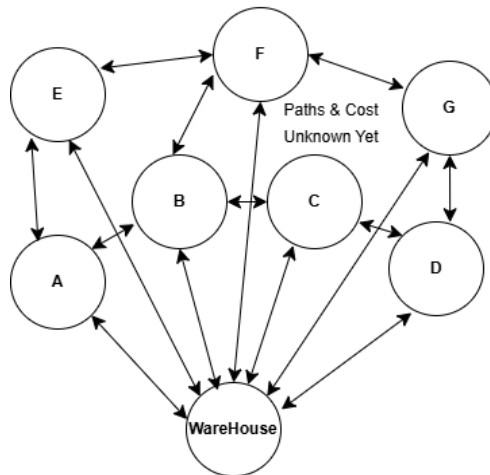
Overall, the integration of the Dijkstra algorithm enhances the effectiveness of the Flood Emergency Delivery System by optimising route planning, reducing delivery delays, and improving the responsiveness of emergency operations. This contributes to a more reliable and efficient disaster management system, ultimately supporting faster aid distribution to flood-affected communities.

10.0 Appendix

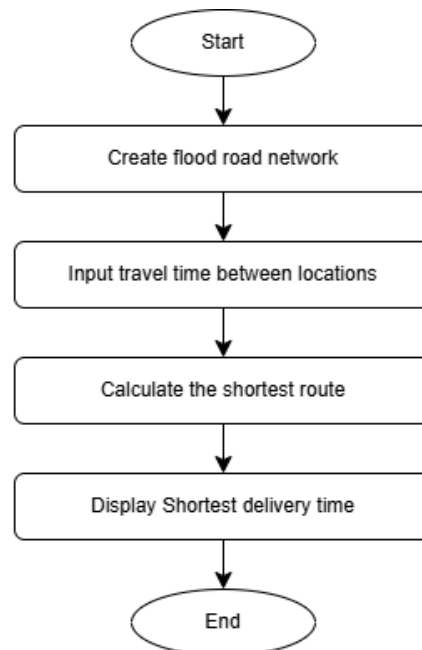
Initial Project Plan

Group Name	First Call		
Members			
	Name	Email	Phone number
	Sio Zixin	SA00287@student.upm.edu.my	012-3546217
	Choo Yu Xuan	SA00012@student.upm.edu.my	011-22769012
	Chai Shun Ying	224414@student.upm.edu.my	012-9803321
	Ng Yu Hui	223821@student.upm.edu.my	011-1677856
Problem scenario description	This project focuses on a flood emergency delivery system involving a warehouse and seven affected villages. Due to flooding, road conditions become unpredictable and inefficient, requiring an optimized routing system to ensure fast delivery of emergency supplies. The system is modelled using a weighted graph where nodes represent locations and edges represent travel routes.		
Why it is important	During flood disasters, delays in emergency supply delivery can severely impact affected communities. An optimized routing system helps reduce travel time, improves resource allocation, and supports efficient disaster response planning.		
Problem specification	The system is modelled as a weighted graph: 1. Nodes = warehouse + villages 2. Edges = roads connecting locations 3. Weights = travel time (minutes) The objective is to find the shortest path from a single source (warehouse) to all villages using graph-based optimization.		
Potential solutions	1. Sorting-based approaches (not suitable for network structure) 2. Divide and Conquer (not applicable for shortest path dependency) 3. Dynamic Programming (limited applicability for general graph routing) 4. Greedy Algorithm (Dijkstra selected as greedy-based approach)		
Sketch (framework,	Framework/ Graph Diagram		

flow,
interface)



Flow



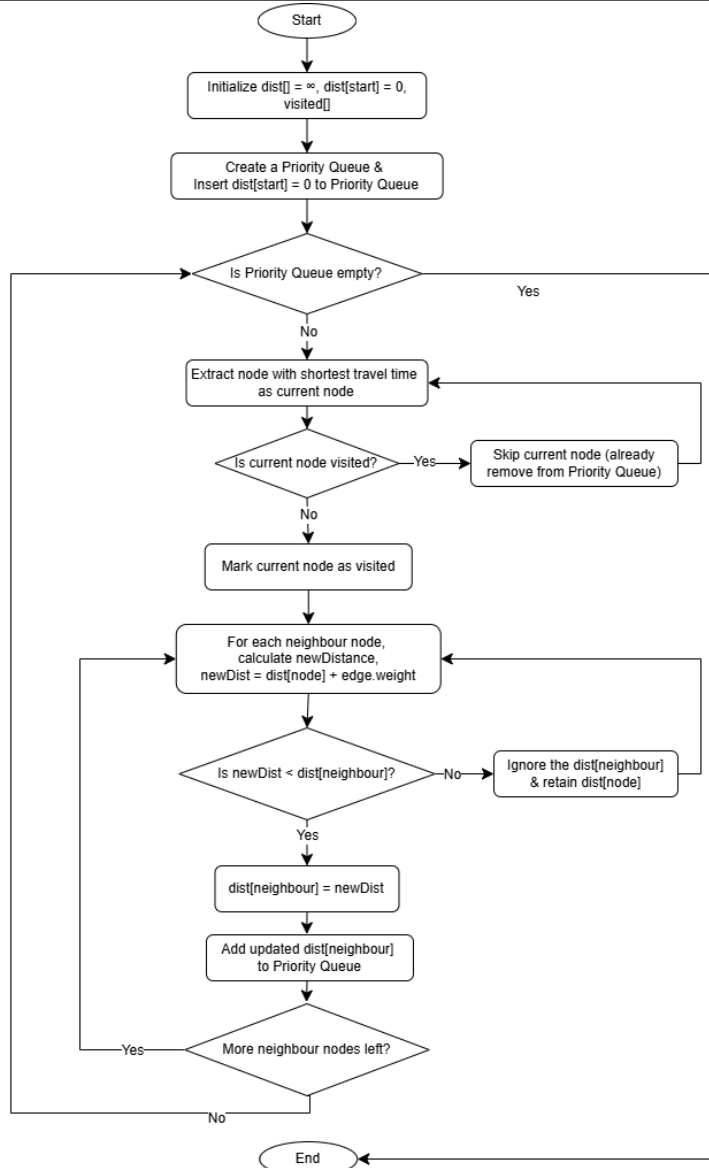
Interface

	<pre> Program should show delivery time per village: Warehouse -> ____ min Village A -> ____ min Village B -> ____ min Village C -> ____ min ... (exact format / values TBD) </pre>
--	---

Project Proposal Refinement

Group Name	Group 7											
Members	<table><tr><th>Name</th><th>Role</th></tr><tr><td>Sio Zixin</td><td>Graph design and network modeling</td></tr><tr><td>Choo Yu Xuan</td><td>Report writing and documentation</td></tr><tr><td>Chai Shun Ying</td><td>Algorithm implementation</td></tr><tr><td>Ng Yu Hui</td><td>Diagram design and presentation slides</td></tr></table>		Name	Role	Sio Zixin	Graph design and network modeling	Choo Yu Xuan	Report writing and documentation	Chai Shun Ying	Algorithm implementation	Ng Yu Hui	Diagram design and presentation slides
	Name	Role										
	Sio Zixin	Graph design and network modeling										
	Choo Yu Xuan	Report writing and documentation										
	Chai Shun Ying	Algorithm implementation										
	Ng Yu Hui	Diagram design and presentation slides										
Problem statement	Flood conditions disrupt normal transportation routes, making emergency delivery inefficient. Therefore, an optimized system is required to compute the shortest delivery time from a central warehouse to multiple affected villages using a graph-based model.											
Objectives	1. To model the flood network using a weighted graph 2. To implement Dijkstra’s Algorithm for shortest path computation 3. To determine optimal emergency delivery routes 4. To analyze time complexity and correctness of the solution											
Expected output	1. Shortest delivery time from warehouse to each village 2. Optimal routing paths using Dijkstra’s Algorithm 3. Display of minimum travel time results for all nodes											
Problem scenario description	This project focuses on a flood emergency delivery system involving a warehouse and seven affected villages. Due to flooding, road conditions become unpredictable and inefficient, requiring an optimized routing system to ensure fast delivery of emergency supplies. The system is											

	modelled using a weighted graph where nodes represent locations and edges represent travel routes.
Why it is important	During flood disasters, delays in emergency supply delivery can severely impact affected communities. An optimized routing system helps reduce travel time, improves resource allocation, and supports efficient disaster response planning.
Problem specification	The system is modelled as a weighted graph: 1. Nodes = warehouse + villages 2. Edges = roads connecting locations 3. Weights = travel time (minutes) The objective is to find the shortest path from a single source (warehouse) to all villages using graph-based optimization.
Potential solutions	1. Sorting-based approaches (not suitable for network structure) 2. Divide and Conquer (not applicable for shortest path dependency) 3. Dynamic Programming (limited applicability for general graph routing) 4. Greedy Algorithm (Dijkstra selected as greedy-based approach)
Sketch (framework, flow, interface)	Framework/ Graph Diagram <pre> graph TD WH[WareHouse 0 min] -- 30 --> A[A 14 mins] WH -- 8 --> B[B 8 mins] WH -- 18 --> C[C 15 mins] WH -- 25 --> D[D 15 mins] WH -- 22 --> E[E 19 mins] WH -- 11 --> F[F 25 mins] WH -- 10 --> G[G 30 mins] A -- 9 --> C B -- 6 --> A B -- 15 --> C C -- 5 --> D C -- 13 --> F D -- 8 --> E E -- 9 --> F F -- 10 --> G G -- 20 --> C </pre> Flow



Interface

===== SHORTEST PATH RESULT =====

Village	Shortest Path	Time Taken (min)
Warehouse	Warehouse	0
Village A	Warehouse -> Village B -> Village A	14
Village B	Warehouse -> Village B	8
Village C	Warehouse -> Village C	15
Village D	Warehouse -> Village B -> Village D	15
Village E	Warehouse -> Village B -> Village E	19
Village F	Warehouse -> Village F	25
Village G	Warehouse -> Village G	30

Methodology		
	Milestone	Time
	Scenario selection and problem definition	wk10
	Graph design and algorithm selection	wk11
	Code implementation and debugging	wk12
	Analysis (correctness and complexity)	wk12
	Report finalization and presentation	wk12

Project Progress

Milestone 1	Scenario selection and problem definition			
Date (week)	Week 10			
Description/ sketch	Scenario selection and problem definition were completed. The flood emergency delivery system was identified, and the initial graph-based model (warehouse + 7 villages) was confirmed.			
Role	Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui
	Scenario design	Background research	Initial report drafting	Meeting coordination

Milestone 2	Graph design and algorithm selection											
Date (week)	Week 11											
Description/ sketch	Graph design was completed and the suitable algorithm (Dijkstra's Algorithm) was selected. The weighted graph structure with nodes, edges, and weights was finalized.											
Role	<table><tr><td>Sio Zixin</td><td>Choo Yu Xuan</td><td>Chai Shun Ying</td><td>Ng Yu Hui</td></tr><tr><td>Graph modelling</td><td>Algorithm selection</td><td>Report structuring</td><td>Diagram preparation</td></tr></table>				Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui	Graph modelling	Algorithm selection	Report structuring	Diagram preparation
Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui									
Graph modelling	Algorithm selection	Report structuring	Diagram preparation									

Milestone 3	Code implementation and debugging			
Date (week)	Week 12			
Description/ sketch	Java implementation of Dijkstra's Algorithm was completed using adjacency list and priority queue. Debugging and testing of graph connections were performed.			
Role				
	Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui
	Graph implementation	Debugging support	Coding (Dijkstra)	Testing results

Milestone 4	Analysis (correctness and complexity)											
Date (week)	Week 12											
Description/ sketch	Analysis of correctness and time complexity was completed. The system output was verified using sample inputs and shortest path results.											
Role	<table><tr><td>Sio Zixin</td><td>Choo Yu Xuan</td><td>Chai Shun Ying</td><td>Ng Yu Hui</td></tr><tr><td>Result verification</td><td>Complexity analysis</td><td>Report writing</td><td>Output validation</td></tr></table>				Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui	Result verification	Complexity analysis	Report writing	Output validation
Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui									
Result verification	Complexity analysis	Report writing	Output validation									

Milestone 5	Report finalization and presentation			
Date (week)	Week 12			
Description/ sketch	Final report was completed and presentation slides were prepared. The full project documentation and online portfolio were finalized for submission.			
Role				
	Sio Zixin	Choo Yu Xuan	Chai Shun Ying	Ng Yu Hui
	Final coding review	Report editing	Overall formatting	Slide preparation